Figure 13.12 A complete Razor Pages request for the list of to-dos in the Simple category

By now, you may be thinking this whole process seems rather convoluted. So many steps to display some HTML! Why not allow the application model to create the view directly, rather than have to go on a dance back and forth with the page handler method? The key benefit throughout this process is the *separation of concerns*:

- The view is responsible only for taking some data and generating HTML.
- The application model is responsible only for executing the required business logic.
- The page handler (controller) is responsible only for validating the incoming request and selecting which response is required, based on the output of the application model.

By having clearly defined boundaries, it's easier to update and test each of the components without depending on any of the others. If your UI logic changes, you won't necessarily have to modify any of your business logic classes, so you're less likely to introduce errors in unexpected places.

The dangers of tight coupling

it's generally a good idea to reduce coupling between logically separate parts of your application as much as possible. This makes it easier to update your application without causing adverse effects or requiring modifications in seemingly unrelated areas. Applying the MVC pattern is one way to help with this goal.

As an example of when coupling rears its head, I remember a case a few years ago when I was working on a small web app. In our haste, we hadn't decoupled our business logic from our